

節 2 / 共 4 節

Harness Engineering

怎麼把 agent 調成我要的形狀？

節 1 只用到了 context 的第一種來源

這一節接上第二、第三種。

已經知道的

- LLM 是在猜，會流暢地講錯
- 它只看得到 context —— 但 agent 會自己去填
- 四個層次：prompt / context / harness / loop

已經做到的

- agent 跑起來了
- 看清楚了它自作主張什麼

這一節要接上的

節 1 用的是第一種：當下打的字。

- **常駐的設定檔** → 開新 session 就知道規矩
- **工具抓回來的** → 能碰到外部系統
- **自動被檢查** → 做完就驗

模型是你選的，
harness 是你做的。

你不能控制模型腦袋裡發生什麼，但你可以完全控制它能碰到什麼、做完要通過什麼檢查。

2-A

Harness 全景圖

一個 agent 由哪些零件組成？

一個 agent 拆開來長這樣

AGENTS.md

每次都載入的規矩

Skills

用到才載入的 SOP

Tools / MCP

真的去執行的手腳

迴路 / 守衛

做完自動檢查、危險的事前擋下

Subagents

分身去做髒活（只講不做）

↑ 以上全部都是你寫的

↓ 這個你只能選，不能改

模 型

這些零件的差別，只有三個維度

零件	何時載入	誰執行	能不能強制
AGENTS.md	每次	模型自己	✗ 只是請求
Skill	用到才載入	模型自己	✗ 只是請求
Tool / MCP	每次（工具清單）	外部系統	✓ 做了就是做了
迴路（事後）	做完之後	你的腳本	✓ 沒過就叫它重做
守衛（事前）	工具執行前	你的腳本	✓ 根本不讓它跑

重點 最右邊那一欄是這一節最重要的分野。前兩個是「拜託模型」，後兩個是「程式強制」。

2-B

AGENTS.md

每次啟動就自動載入的那份規矩。

專案的規矩，每次對話都要重講一遍

這專案用 pnpm、註解寫繁中、不要用 any

—— 開新對話，再打一次

AGENTS.md = 每次都會自動載入的一段話

通稱是「初始輸入檔」，但今天我們就叫它 AGENTS.md。

- agent 啟動時會**自動讀進 context** 的檔案
- 等於「你不用每次重打的那段話」

規矩寫在哪	它是哪一種 context
你打的 prompt	當次 —— 這次要做什麼
AGENTS.md	持久 —— 這個專案永遠的規矩

實務 這是 context engineering **最便宜的做法**：寫一次，之後每次對話都自動生效。

同一個概念，每個工具取不同的名字

初始輸入檔是這樣沒錯 —— 但 **rules** 這個詞不是。

工具	初始輸入檔	備註
Claude Code	CLAUDE.md	它不讀 AGENTS.md —— 用 @AGENTS.md 引進來（下一頁）
Codex	AGENTS.md	跨工具的事實標準，最多人用
今天用的 pi	兩個都讀	全域 → 各層父目錄 → 當前目錄；AGENTS.override.md 取代 而非疊加

重點 到這裡為止概念相同，只是檔名不同 —— 要記的是「有一個檔案會自動載入」。

注意 但 **rules** 是同名不同物

誰的 rules	實際上是什麼	性質
Claude Code、Cursor	.claude/rules/*.md，paths: glob 決定何時載入	餵 context 拜託
Codex	execpolicy：指令 allow / prompt / forbidden	擋執行 強制

實務作法：單一真實來源

- 用 AGENTS.md 寫**唯一一份**內容
- 其他工具要的檔名，寫一行指過去就好

See @AGENTS.md

CLAUDE.md 的全部內容

注意 不要維護兩份。兩份一定會不同步，然後你會花一個下午 debug 「為什麼這個工具聽話另一個不聽」。

作用域：三層，但它們是疊加，不是覆蓋

而且三層的載入時機不一樣。

層級	放什麼（例子）	什麼時候載入
全域（你的家目錄）	我個人 的習慣，跨所有專案 回答用繁體中文	開新 session
專案根目錄	這個專案 的規矩 用 pnpm、禁止 any	開新 session
子目錄	這個模組 的特例 legacy/ 底下不要重構	它讀到那層的檔案時 才載入

重點 三層是**全部串接**進 context，從外層排到內層 —— **不是覆蓋**，也沒有「範圍小的贏」這種保證。真的矛盾了，它可能隨便挑一個。

實務 Lab 2 進階題 三層都寫一份、故意讓它們打架 —— **要驗的是「有沒有穩定的贏家」**。

該寫什麼

- ✓ **怎麼跑測試、怎麼 build** —— 最常被猜錯的一項
- ✓ **非顯而易見的慣例** —— 「我們的 API 一律回 snake_case」
- ✓ **專案術語** —— 「訂單」指的是 PurchaseOrder，不是 Order
- ✓ **明確的禁止事項** —— 「不要動 migrations/」 「不要自己加套件」
- ✓ **改 A 就得一起改 B** —— 「新增一種付款方式，要同時改 x、y、z」

實務 判斷法則：新同事第一天上工會口頭交代的，就該寫進去。

不該寫什麼

- **X 程式碼結構** —— 它自己讀得到，而且你寫的很快就過期
- **X git log 查得到的東西** —— 改動歷史、誰改的、為什麼
- **X 專案背景故事** —— 「本專案源於 2023 年的一場討論…」
- **X 行銷語言** —— 「我們致力於打造世界級的…」

重點 第一條要跟上一頁分清楚：**「有哪些檔案」不用寫**（它讀得到、而且會過期）；**「改這個就得一起改那個」要寫**（那是約定，不在檔案裡）。

注意 每一行都佔 context，而且每一次對話都佔。寫成 500 行只會稀釋重要的那幾條。

怎麼寫，跟寫什麼一樣重要

三條是實測出來的，不是風格偏好。

規矩	為什麼
一行一件事	一行塞三件事，它會只做到其中一件。條列與表格比整段敘述有效
具體到可以驗證	「用兩格縮排」有效，「把格式寫好」沒有效
不要超過 200 行	Anthropic 官方對 CLAUDE.md 的建議值。越長，遵守率越差

重點 超過 200 行怎麼辦？**不是刪掉，是搬家**——偶爾才用的搬去 skill，只跟某個目錄有關的搬去那個目錄。

注意 兩條規矩互相矛盾時，**它會隨便挑一條**，而且不會告訴你。定期回去清。

AGENTS.md 不是 README。

它是給新同事的交接便條。

交接便條是「你會踩到的坑在這裡」，不是「本專案簡介」。

起手式：讓它自己寫一份

- Claude Code / Codex 有 `/init`，掃過專案生成初版
- pi 沒有這個指令，直接叫 agent 掃一遍寫一份也一樣
- **但一定要人工砍掉**—— 它生出來的多半是程式碼結構摘要，正是不該寫的那類

它生成的	你該做的
50 行專案結構描述	全部刪掉
「使用 TypeScript 開發」	刪掉，它看得到
「測試指令： <code>pnpm test</code> 」	留著，這才是重點

最重要的一課

你寫了，**不代表它會照做**。

- 能力較弱的模型（例如我們今天用的免費模型）尤其如此
- 規則越多，被忽略的機率越高
- 你**沒有辦法**從它的回覆判斷它有沒有遵守，只能檢查產出

重點 所以下一個問題是：**能不能用程式強制？** → 那是 2-F 的迴路與守衛。先把「拜託模型」這一半講完。

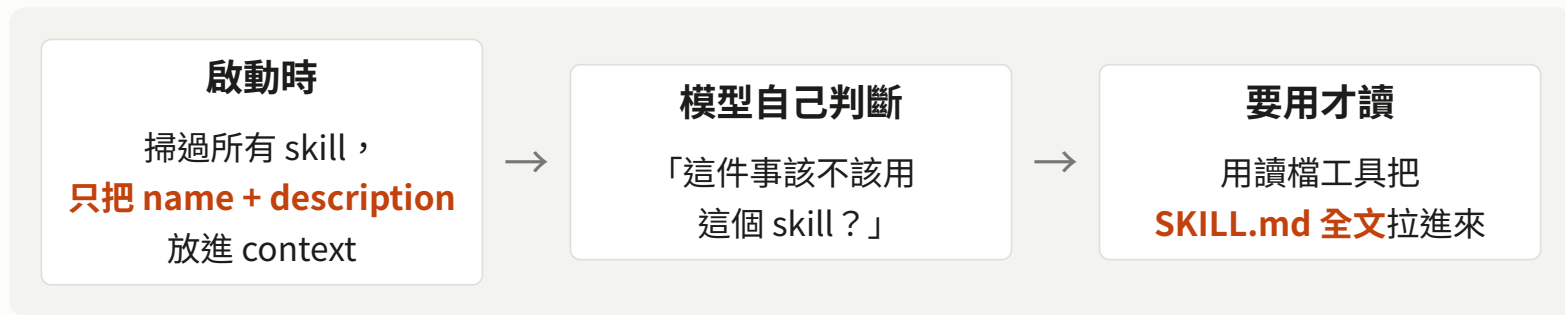
2-C

Skills

用到才載入的做事說明書。

Skill 的重點是「漸進式揭露」

這個機制決定了它的成本，也決定了它適合放什麼。



重點 所以：一百個 skill 的常駐成本，等於一百行描述 —— 而不是一百份文件。這就是它能一直長大而不吃掉 context 的原因。

注意 代價在第二格：判斷是模型做的，它不一定會讀。

什麼適合寫成 skill

適合寫成 skill 的	例子
偶爾才跑一次的流程	產生本專案格式的 commit message
步驟多、需要照順序	發版流程：改版號 → 更新 changelog → 打 tag
有固定產出格式	寫一份符合公司樣板的技術文件
某個外部系統怎麼用	內部資料庫的表結構、連線指令、踩過的雷

重點 最後一列等一下會再回來 —— 它就是「skill 還是 MCP」那個問題的核心。

一個 skill 長什麼樣

一個資料夾，裡面一個 SKILL.md。其他都是選配。

```
---
name: commit-msg
description: 產生符合本專案格式的 commit message。當使用者要
commit 的時候使用。
---

# 產生 commit message

## 步驟
1. 跑 `git diff --staged`
2. 判斷類型：feat / fix / refactor
3. 照格式寫，只輸出訊息本身
```

commit-msg/SKILL.md

檔案結構

```
my-skill/
├── SKILL.md      ← 只有這個必要
├── scripts/     ← 它可以去跑的腳本
├── references/  ← 需要時才讀的細節
└── assets/      ← 樣板、範例檔
```

必填欄位	規則
name	小寫、數字、連字號
description	做什麼 + 什麼時候用

注意 缺 description 的 skill 不會被載入 —— 其他違規只會警告。

description 決定它會不會被用

常駐在 context 裡的只有 name 和 description，全文要它自己去讀。

主流寫法	「當…的時候使用」 + 它會做什麼 —— 觸發條件擺前面
✗ 不好	處理 PDF。 ← 只講功能，沒講什麼時候該用
✓ 好	當使用者要處理 PDF 的時候使用：抽出文字與表格、填表單、合併多個 PDF。

重點 description 是寫給模型看的，不是寫給人看的。它是模型判斷「現在這件事 該不該用這個 skill」的**唯一依據** —— 所以要寫的是**觸發條件**，不是功能簡介。

注意 寫得模糊，它就**不會選中**這個 skill —— 而且**沒有任何錯誤訊息**，因為對系統來說**什麼事都沒發生**。

AGENTS.md VS Skill

	AGENTS.md	Skill
載入時機	每次都載入	用到才載入全文
context 成本	全文持續佔用	只有 name + description 常駐
適合	一直都要遵守的約束	偶爾才跑一次的流程
判斷法則	「每次都要」	「偶爾才做」

實務 直接的推論：**AGENTS.md 太長的時候，把偶爾用到的部分抽成 skill**。這是實務上最常見的一次重構。

注意 但 **skill 的常駐成本不是 0** —— 每多一個 skill，就多一行描述一直待在 context 裡。

2-D

Lab 2 : AGENTS.md + Skill

寫規範，然後量它到底聽不聽話。

Lab 2 AGENTS.md + Skill

60 分鐘

目標 親手證明「有沒有寫規範」會產生不同的程式碼，以及「寫了也不一定照做」。

素材： labs/lab2-agents-md/grades/ —— 能跑但很醜的成績計算工具，只有兩個檔。

1. **基準** 不給任何規範，讓 agent 重構。記錄它自作主張了什麼
2. **加約束** 寫一份 AGENTS.md，用**完全相同的指令**再跑一次，在 **VS Code** 比較兩次的改動
3. **測遵守率** 同一份 AGENTS.md 連跑 3 次，統計每條規則被遵守幾次
4. **抽成 skill** (進階) 把「產生 commit message」寫成 skill，用 /skill: 強制呼叫

注意 步驟 3 如果遵守率不是 100% —— **這不是 lab 失敗，這正是本 lab 要教的事。** 把數字記下來。

先說：這個 lab 不用你看懂程式在算什麼

七條規則裡有六條是「看得出來」的，不是「想得出來」的。

規則	違規長這樣	遵守長這樣
R1 不用 any	<code>function calc(d: any)</code>	<code>function calc(d: Course)</code>
R2 匯出函式小駝峰	<code>export function Load()</code>	<code>export function loadData()</code>
R3 用 template literal	<code>"avg: " + s.avg</code>	<code>avg: \${s.avg}</code>
R4 數字抽成常數	<code>score = hw * 0.3</code>	<code>score = hw * W_HOMEWORK</code>
R6 匯出函式有說明	(上面一片空白)	<code>// 算出總分與等第</code>
R7 完整 JSDoc	(沒有 <code>/** */</code> 區塊)	<code>@param</code> 每個參數一行 + <code>@returns</code>

重點 用眼睛找這六種樣子就夠了，不用懂它怎麼算成績 —— 而且 `check.ts` 會幫你數。

注意 R5「註解要說明為什麼」不在上面，那條只能人工看。Lab 4 會回來講為什麼。

Lab 2 記錄表

步驟 3 的統計。這張表的數字，Lab 4 會用到。

規則	起始	第 1 次	第 2 次	第 3 次
R1 禁止使用 any	5			
R2 匯出函式小駝峰、動詞開頭	1			
R3 組字串用 template literal	5			
R4 魔術數字要抽成常數	7			
R6 匯出函式要有一行說明	2			
R7 JSDoc 要有 @param / @returns	3			
R5 註解要說明「為什麼」	人工看			

在 grades/ 跑 `node ../../lab4-loop/check.ts .`，
它會幫你數前六條，共 23 個。

重點 等一下 Lab 4，用迴路把這些數字壓下去。

2-E

工具

從它本來就有的，到你自己接上去的。

它本來就有的工具，先用好

很多「模型不知道」，其實是「你沒讓它去查」。

讀檔	Cargo.lock、package.json —— 實際裝的版本 ，不是它記得的版本
跑指令	cargo check、npm view、跑測試 —— 結果是事實 ，不是回憶
搜尋網路	新套件、改過的 API、昨天發的 release

重點 節 1 地圖的第一列（版本落後、API 搬家），**靠這三件事就解掉了**。

注意 所以「換更強的模型」通常不是答案。**答案是讓它去查**。

MCP 是接到外部系統的標準插頭

內建工具碰不到的東西，用 MCP 接。

- 一個**標準插頭**，讓 agent 能接到**內建工具碰不到**的系統
- 資料庫、瀏覽器、公司內部 API、檔案系統、雲端服務…
- 重點在「**標準**」：寫一次 server，所有支援 MCP 的工具都能用

重點 **MCP 本身是一份說明書** —— 它告訴 agent 「外面有哪些功能、參數長什麼樣、要怎麼呼叫」。

注意 但**說明書後面接的是真的會動的東西**。接上一個能改資料庫的 MCP，就等於給了 agent 改資料庫的權限。

Skill vs MCP：實務上到底怎麼選

	Skill + 內建工具	MCP
常駐 context	只有 name + description	整包工具清單，每次都在
誰執行	模型用它已經有的工具（bash、讀檔）	外部 server
做得到什麼	凡是 bash 做得到的 —— 可組合、可 pipe	只有 server 開出來的那幾個
跨工具	綁這個 agent	✓ 標準，換工具照用
要跑服務	不用	要

注意 「碰不碰得到外面」不是分界線 —— 給了 bash，skill 一樣連得上資料庫。

重點 真正的分界是**你想給它多少權力**：要**更多能力**→skill+CLI，要**更少能力**→MCP。

一個具體的例子：要讓 agent 操作內部資料庫

同一件事，兩種做法，差別不在方便，在權力。

	寫成 skill	做成 MCP server
內容	表結構、連線指令、踩過的雷	開出 query(sql) 這類工具
agent 怎麼做	自己組 SQL，用 bash 跑 psql	呼叫你開的那幾個工具
彈性	很高 ，能 join、能 pipe 進別的指令	只有你開的那幾種
它能 DROP TABLE 嗎	能。你給了 psql 就給了全部	不能，除非你開這個工具
context 成本	平常 0	工具清單每次都佔

重點 「**我有完整權限**」不是選 skill 的理由 —— **那正是最該用 MCP 的情況**，因為 agent 也就有完整權限。

實務 唯讀、自己用、要自由組合 → **skill**。會寫入、多人共用、要稽核 → **MCP 或一層守衛**。

MCP 在 pi 上要怎麼用？

這一頁是「概念相同、實作各異」最好的例子。

pi 官方明文寫著：它**刻意不內建** MCP、subagent、permission popup、plan mode。

- 這不是缺陷，是設計取捨：核心極小，其他靠 extension
- **「這個功能怎麼用」取決於手上是哪個工具**

實務 等一下的 Lab 3 我們**自己寫一個工具**，跳過 MCP 那一層。

重點 課後 附錄有可跑的 Python MCP server 與探測程式。

labs/appendix-mcp/python/ 完全不需要 pi，兩支 Python 檔互相對話。

2-F

Lab 3：給 agent 一雙手

第一個「真的會執行」的東西。

Lab 3 給 agent 一雙手

目標 親手註冊一個工具，然後證明 **description** 寫得好不好，決定它會不會被用。

素材：labs/lab3-tools/。40 行 extension，**三個 TODO 全是「字」**，免安裝。

1. **對照組** 不載入工具，問「想去社辦寫 code，E205 方便嗎？」→ 它反問 E205 是什麼
2. **載入工具** `pi -e ../club-tool.ts` → **它多半還是答不出來**
3. **改好三個 TODO** 名字、description、參數說明 → `node verify.ts`
4. **進階** 再加一個「借教室」的工具 —— 從「查」變成「做」

注意 一定要在 **workspace/ 裡面跑**，不然 agent 會直接去讀檔案。

重點 驗收 3 人 / 器材長 / 21:00 **只存在工具的資料裡**。

為什麼第 2 步會失敗

模型看得到的只有這三樣字。

```
name: "tool_a",  
description: "一個工具。",  
parameters: { room: { type: "string", description: "參數" } },
```

問法	爛 description 的結果
「E205 現在有人嗎？」	會呼叫 —— 問題裡有 E205，參數說明裡也有，字面對上了
「想去社辦寫 code，方便嗎？」	x 不呼叫 —— 它得靠 description 才知道這件事該用這個工具

重點 所以驗收問題**故意問得間接**。**不要改成直接問**，改了兩組就看不出差別。

2-G

Loop Engineering

取代「一直按 enter 的那個人」。

一個迴路只有三個零件

設計迴路，就是決定這三件事。

	等一下的 Lab 4	「把題目全解完」的迴路
工作從哪來	就這一個重構任務	題目網站上還沒解的題
每一輪做什麼	改 → 跑檢查 → 錯誤餵回去	挑一題 → 解 → 送出 → 看結果
什麼時候停	檢查過了，或重試 3 次	清單空了

重點 最容易被忽略的是最後一列。沒有明確的停止條件，agent 會一直跑下去 —— 或者一輪就停在你沒預期的地方。

注意 停止條件必須**客觀可判定**：編譯過了、測試綠了、清單空了。「做得夠好了」不算。

Lab 4 讓它自己修錯

30 分鐘

目標 把 Lab 2 那張表裡「機器檢查得到」的規則，遵守率明顯拉上去。

問題回顧： AGENTS.md 明明寫了「禁止 any」，它還是用了。核心就這四步：

```
runAgent(TASK); // 做
const c = run("node", ["check.ts", TARGET]); // 檢查
if (c.code === 0) process.exit(0); // 過了就收工
runAgent(`違規如下：${c.out} 請修好`, true); // 餵回去 → 再做
```

```
node loop.ts "把 calc.ts 與 report.ts 依照 AGENTS.md 重構"
```

注意 畫面會空白很久，正常 —— pi -p 做完才輸出。⌚ 秒數在跳就是還活著。

重點 驗收 錯誤訊息被**自動**餵回去，遵守率明顯上升。loop.ts 的註解值得打開看。

Lab 4 的第二個問題（比第一個重要）

check.ts 實作了 R1、R2、R3、R4、R6、R7 —— 唯獨沒有 R5。為什麼？

規則	程式擋得住嗎
R1 禁止使用 any / R3 template literal	✓ grep 就抓得到
R2 匯出函式命名	✓ 一行 regex
R7 JSDoc 要有 @param / @returns	✓ 也是 grep —— 對照參數名就好
R4 魔術數字要抽成常數	勉強 會有誤判
R5 註解要說明「為什麼」	✗ 沒有任何 linter 擋得住

重點 R7 跟 R5 都在管註解，差別在哪？R7 管的是**形狀**（有沒有寫 @param），R5 管的是**意義**（寫得有沒有價值）。

注意 程式檢查得到形狀，檢查不到意義。設計規範時就要往「可被檢查」的方向設計。

先說一下 .env 是什麼

幾乎每個專案都有一個，而且它是 agent 最不該碰的那個檔。

```
DATABASE_URL=postgres://user:  
password@localhost:5432/db  
OPENCODE_API_KEY=sk-你的真 key  
SESSION_SECRET=change-me
```

一個典型的 .env

你 Lab 0 拿到的那把 key，在真的專案裡就是放這裡。

它的三個特性

- **放的全是機密**：資料庫密碼、API key、簽章用的 secret
- **一定被 .gitignore 擋掉**，所以不會進版本控制
- **但它就躺在專案根目錄**，agent 一個 read 就拿到

注意 明天 Lab 6 你會親手把自己的 key 從網頁挖出來。**這裡是同一件事的另一面。**

重點 所以問題變成：**你要怎麼確保 agent 不會去讀它、更不會把內容貼到某個地方？**

有些事情不能事後修

它把 `.env` 讀出來貼到某個地方之後，你修不回來。

	loop.ts (事後)	守衛 (事前)
何時介入	agent 整輪跑完後	工具執行前
能不能阻止	不能，只能叫它改	能，根本不讓它跑
適合	品質問題：風格、型別	安全問題：機密、刪檔、對外送出的

```
pi.on("tool_call", async (event) => {           // 執行「前」
  if (findSecretRef(event.input))
    return { block: true, reason: "改讀 env.sample" };
});
```

重點 判斷法則：**修得回來的用事後，修不回來的用事前。**

守衛也是程式，也會有洞

這是進階組今天最好玩的十分鐘。

guard/env-guard.ts 第一版的樣式是這樣寫的：

```
/(^|[/\]\])\.env(\.|$)/i // 要求 .env 前面是斜線或開頭
```

然後 `cat .env | head` **直接穿過去** ——
因為在 shell 指令裡，`.env` 前面是**空格**，不是斜線。

- guard/test.ts 裡有 10 種攻擊寫法，其中兩種就是這樣漏掉的
- **你的任務：再想幾個繞法出來**

重點 守衛沒有測試，你不會知道它漏了什麼 —— 而且**不會有任何錯誤訊息告訴你**。

Prompt 是請求。

迴路是強制。

跟安全、跟品質有關的規則，用迴路擋。不要用拜託的。

2-H 小測：這六件事該用哪個零件？

舉手。四個選項：AGENTS.md / skill / 工具 / 迴路。

	情境	答案
1	每次 commit 訊息都要用中文	
2	要查公司內部的訂單資料庫	
3	絕對不能碰 prod/ 目錄	
4	每季做一次的資安稽核流程	
5	產出必須通過型別檢查	
6	專案的術語表	

重點 答完之後，我們把 2-A 那張「何時載入 / 誰執行 / 能不能強制」的表**當場填滿**。

附錄 Subagent / Agent team / Workflow

只講不做 —— 這三個 pi 都沒有內建，是別的工具（Claude Code 等）的能力。

	做什麼	什麼時候用
Subagent	把獨立的髒活丟給分身做	不想讓一堆搜尋結果污染主對話
Agent team	多個角色分工：寫的、找 bug 的、驗證的	需要互相檢查 的時候
Workflow	寫死的流程，不讓模型自由發揮	需要可重現、可稽核、控成本

重點 Workflow 的判斷法則值得記：當「每次結果都一樣」比「每次都很聰明」更重要時，就寫死流程。

注意 pi 核心沒有內建 subagent，要靠 extension 或開多個 instance —— 又一個「概念相同、實作各異」的例子。

agent 現在知道專案的規矩了

節 2 小結

本節重點

- agent 由五種零件組成
- AGENTS.md = 持久 context
- AGENTS.md 不是 README
- skill 按需、AGENTS.md 常駐
- 工具與 MCP 是真的會執行
- **prompt 是請求，迴路是強制**

明天：它還不知道的事

- 社團的內部規定
- 公司的內部文件

以及：怎麼把這一切**放進真的產品裡**。

實務 節 1 那句「你把公告貼上來，我就
能幫你找出」—— **明天把「貼上來」這件
事自動化**。